

Homework 4.1 Report

Tjaš Ajdovec

ta5946@fri.uni-lj.si

Implementing Nelder-Mead Method

We implemented the **Nelder-Mead simplex method** in Python. Each iteration proceeds as follows:

1. Sort the $n + 1$ simplex points by function value: $y_0 \leq y_1 \leq \dots \leq y_n$.
2. Compute the centroid $\bar{x}$ of all points except the worst.
3. **Reflect** the worst point through the centroid with coefficient $\alpha = 1$.
4. If the reflected point is the new best, attempt an **expansion** with coefficient $\gamma = 2$.
5. Otherwise, if it is still worse than the second-worst, attempt a **contraction** (inside or outside) with coefficient $\rho = 0.5$.
6. If contraction fails, **shrink** all points toward the best point with coefficient $\sigma = 0.5$.
7. Terminate when both the simplex diameter and the spread of function values fall below a tolerance.

The implementation is **dimension-independent**: the dimension n is inferred from the starting point as $n = \text{len}(x_0)$, and the initial simplex of $n + 1$ points is constructed as $x_0, x_0 + d e_1, \dots, x_0 + d e_n$, where d is the chosen diameter and $e_1, \dots, e_n$ are the coordinate unit vectors.

Comparing Nelder-Mead with Gradient Descent Methods

We compared Nelder-Mead against six gradient-based methods from Homework GD.2: **GD**, **Polyak GD**, **Nesterov GD**, **AdaGrad**, **Newton** and **BFGS**. The comparison was run on three test functions (Figure 1):

- **Quadratic** $f_a(x, y, z) = (x - z)^2 + (2y + z)^2 + (4x - 2y + z)^2 + x + y$, starting points $(0, 0, 0)$ and $(1, 1, 0)$.
- **Rosenbrock-like** $f_b(x, y, z) = (x - 1)^2 + (y - 1)^2 + 100(y - x^2)^2 + 100(z - y^2)^2$, starting points $(1.2, 1.2, 1.2)$ and $(-1, 1.2, 1.2)$.

- **Beale** $f_c(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$, starting points $(1, 1)$ and $(4.5, 4.5)$.

Nelder-Mead is a **zero-order method** that uses only function values, while the gradient-based methods require first-order information. Newton additionally requires the Hessian and BFGS builds an approximation of it. All methods were run with `max_iter=1000`. Results are shown in Table 1.

Method	s1		s2	
	iter	gap	iter	gap
Newton	1	$< 10^{-10}$	1	$< 10^{-10}$
BFGS	13	$< 10^{-10}$	8	$< 10^{-10}$
Nelder-Mead	113	$< 10^{-10}$	113	$< 10^{-10}$
Polyak GD	354	$< 10^{-10}$	368	$< 10^{-10}$
Nesterov GD	365	$< 10^{-10}$	392	$< 10^{-10}$
GD	1000	10^{-4}	1000	10^{-3}
AdaGrad	1000	10^{-8}	1000	2.5

Method	s1		s2	
	iter	gap	iter	gap
Newton	8	$< 10^{-10}$	33	$< 10^{-10}$
BFGS	1000	$< 10^{-10}$	1000	$< 10^{-10}$
Nelder-Mead	151	$< 10^{-10}$	275	$< 10^{-10}$
Polyak GD	1000	10^{-8}	1000	10^{-5}
Nesterov GD	1000	10^{-3}	1000	10^{-5}
GD	1000	10^{-3}	1000	3.4
AdaGrad	1000	10^{-2}	1000	4.2

Method	s1		s2	
	iter	gap	iter	gap
Nelder-Mead	74	$< 10^{-10}$	77	$< 10^{-10}$
BFGS	15	$< 10^{-10}$	1000	7.3
Newton	1	14.2	13	14.2
Polyak GD	1000	10^{-5}	1000	inf
Nesterov GD	1000	10^{-5}	1000	inf
GD	1000	10^{-2}	1000	inf
AdaGrad	1000	2.5	1000	10^4

Table 1. Iterations and gap $|f(x_i) - f_{opt}|$ for f_a (top), f_b (middle) and f_c (bottom). Values marked inf indicate the iterates diverged to infinity.

Newton converged fastest on f_a and f_b (Figure 2). On the quadratic function it reached the minimum in a **single step**, as

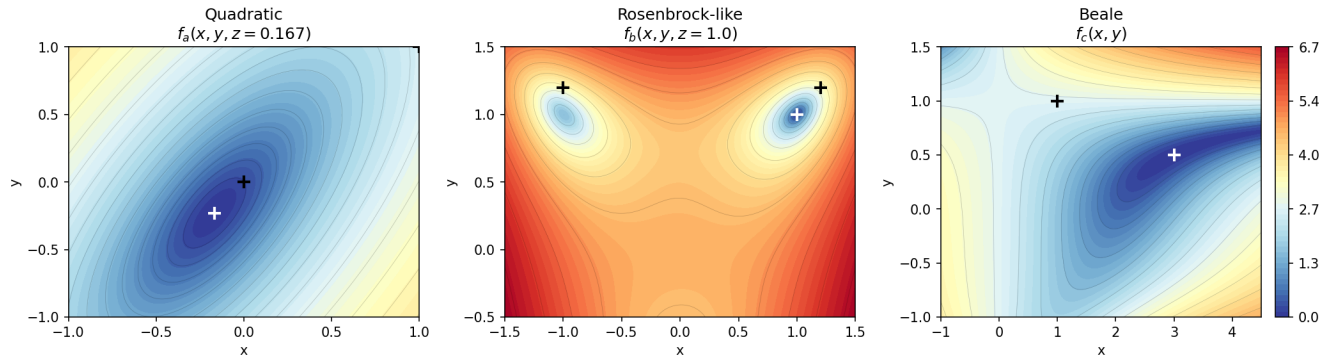


Figure 1. Heatmaps of the three test functions on a 2D slice with z fixed at the optimum (log scale, shared color axis). White cross marks the minimum, black crosses mark the starting points.

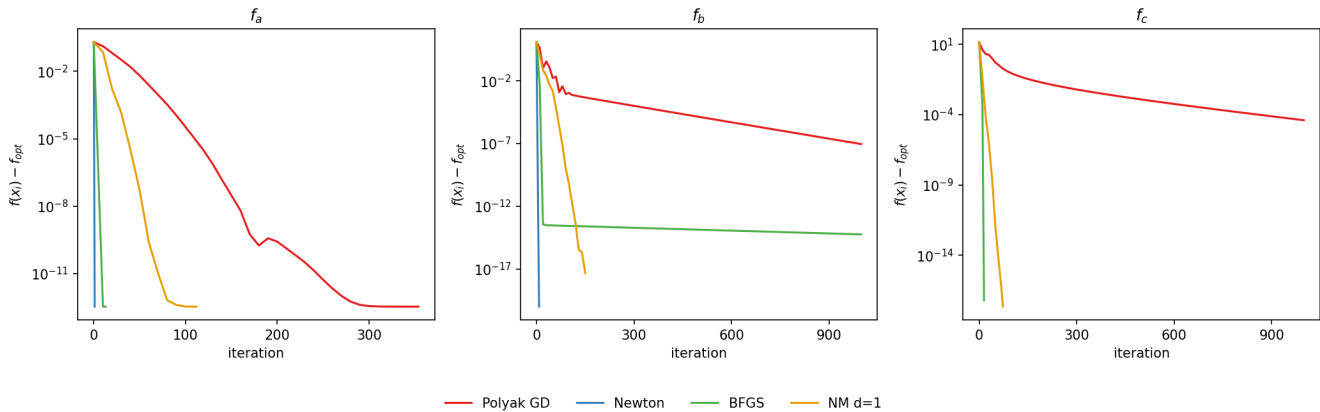


Figure 2. Convergence of selected methods on the three test functions from the easier starting point. Lower is better.

expected for a method with exact second-order information. On f_b , first-order methods hit the iteration limit without fully converging, while Newton and Nelder-Mead both reached the minimum. BFGS also converged on f_b despite hitting the iteration limit.

On f_c , Nelder-Mead was the only method to converge from **both starting points**. From $(4.5, 4.5)$, GD, Polyak GD and Nesterov GD diverged, BFGS got stuck at a wrong point and Newton converged to a local point far from the minimum. This makes Nelder-Mead the most **generally applicable** method, especially when derivatives are unavailable or the starting point is far from the minimum.

Black Box Optimization

We minimized three black-box functions $f_{63210005,i} : \mathbb{R}^3 \rightarrow \mathbb{R}$, $i \in \{1, 2, 3\}$, evaluated through `hw_4.1.win.exe`. Since exact gradients are unavailable, we approximated them using **central finite differences** with $h = 10^{-5}$:

$$\frac{\partial f}{\partial x_j}(x) \approx \frac{f(x + he_j) - f(x - he_j)}{2h}.$$

We compared BFGS with finite-difference gradients against Nelder-Mead with $d = 1$, both from starting point $(0, 0, 0)$

with `max_iter=20`. BFGS was chosen as it builds approximate second-order information without requiring an exact Hessian. The results are shown in Table 2.

Function	BFGS		Nelder-Mead	
	final value	evals	final value	evals
f_1	0.500	36	0.500	43
f_2	0.509	172	0.724	40
f_3	0.500	70	0.500	42

Table 2. Final values and black-box function evaluations. Both methods were run from $(0, 0, 0)$ with `max_iter=20`.

BFGS achieved a **lower final value** on f_2 , while both methods reached the same value on f_1 and f_3 . BFGS uses more evaluations as each iteration requires two black-box calls per dimension for the gradient approximation, plus additional calls for the Armijo line search.

A Toy Solution to a Linear Problem

We solved the following **linear program**:

$$\max x_1 + 3x_2 + 4x_3 \quad \text{subject to} \quad Ax \leq b, x \geq 0,$$

with five inequality constraints and three nonnegativity constraints. Since the objective is linear, it has no curvature and

keeps increasing until it hits a constraint boundary. The optimum therefore always lies at a **vertex** of the feasible region, where exactly $n = 3$ constraints are simultaneously active.

We found all vertices by checking all $\binom{8}{3} = 56$ triples of constraints. For each triple we solved the resulting 3×3 linear system and kept the solution only if it satisfied all remaining constraints. The feasible candidate with the largest objective value is the optimum.

The code found **6 feasible vertices**. The optimal solution is $x = (0, 2.5, 1)$ with objective value

$$x_1 + 3x_2 + 4x_3 = 0 + 3 \cdot 2.5 + 4 \cdot 1 = 11.5,$$

with active constraints $x_1 + 2x_2 + 3x_3 = 8$, $3x_1 + 2x_2 + x_3 = 6$ and $x_1 = 0$.

For n variables and m inequalities the same approach checks $\binom{m}{n}$ combinations, which grows very quickly. For example, $m = 50$ and $n = 10$ already gives over 10^{10} candidates, making the method impractical for larger problems.